

Hexfields: Dominion

- Abschlusspräsentation am 03.06.2025 -



Software Engineering Projekt von

Alexander Horst (Matrnr. 1126225)

Marcel Steinle (Matrnr. 6322372)

Jona Lauber (Matrnr. 9780567)

Gliederung

1. Vision
2. Architektur
3. Tech-Stack
4. Tests und Metriken
5. CI/CD
6. Projektmanagement
7. Learnings
8. Demo

Vision

A dark blue, solid-colored shape that starts from the bottom left corner and extends diagonally upwards towards the right, covering the bottom half of the slide.

Vision

- Mehrspieler-Spiel im Webbrowser
- online zugängliche Alternative zum klassischen Brettspiel *“Siedler von Catan”*
- Mit Freunden oder zufälligen Spielern
- Mit oder Ohne Accounts
- Egal welcher Computer, nur ein Browser nötig



Architektur

A dark blue, diagonal, triangular shape that originates from the bottom-left corner and extends towards the top-right corner, covering the lower half of the image.

Architektur

Codestruktur

- Frontend: Component-basiert
 - `components/` → UI-Elemente
 - `pages/` → Seiten-JSX
 - `public/` → globale Ressourcen
- Backend: Package-basiert
 - `account` → Auth, Login, ...
 - `game` → Spiellogik, Züge
 - `lobby` → Lobby-Verwaltung
 - ... mit `.dto-Packages` (Abstraktion)



Bedingt durch React Framework

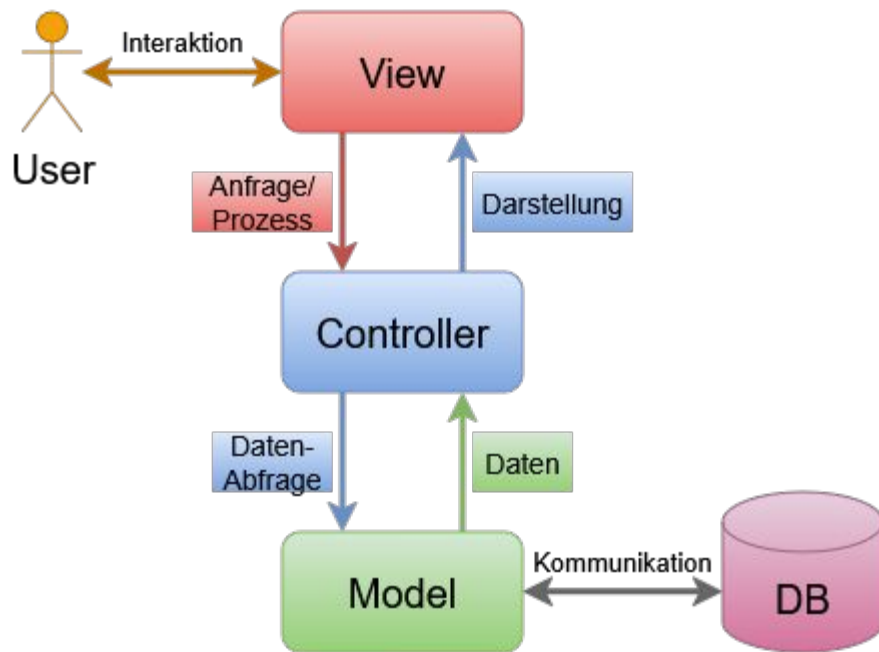


Bedingt durch Repository-Pattern

Architektur

Systemarchitektur

- Nach **MVC-Modell** vorgegangen:
 - View auf einer Webseite
→ React TypeScript
 - Controller auf dem Server
→ Java Spring Boot
 - Model mit der Datenbank
→ PostgreSQL



Architektur

Repository-Pattern

- Backend: Repositories repräsentieren Tabellen
 - DB-Anfragen im Code laufen immer über ein Repository ab
 - Repository-Klassen als Schnittstelle zur Datenbank
 - Übernehmen Mapping: Java Objekte ↔ DB-Format
- Frontend: Repository Klasse
 - Per Hooks Zugriff auf Repository
 - Repository wird mit SSE aktualisiert

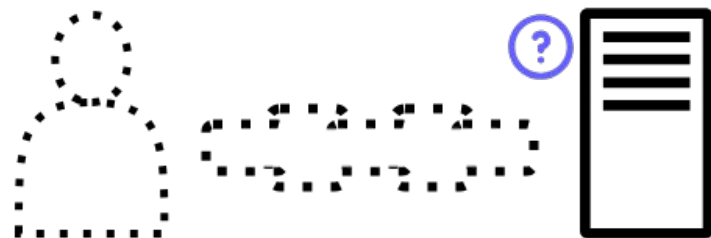
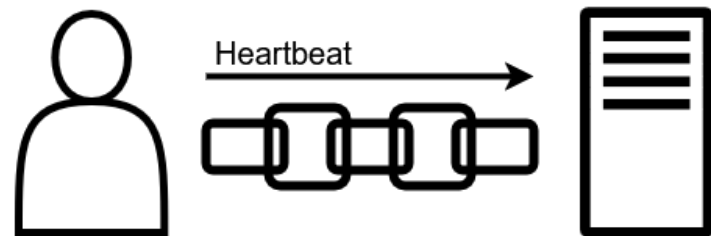
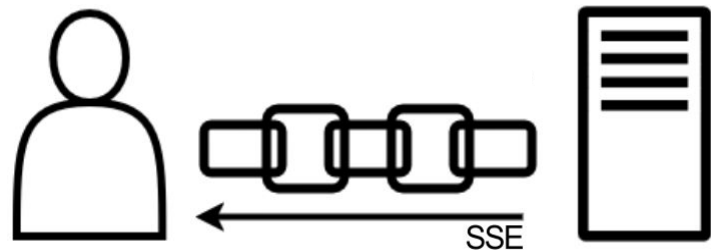


Quelle:
norberteder.com/das-repository-pattern-anhand-eines-beispiels-inkl-tests/

Architektur

SSE & Heartbeat

- SSE-Verbindung zum Datenaustausch
→ keine hohe Übertragungsrate notwendig
→ effizienter als WebSocket
- Client-Heartbeats werden zurückgesendet
→ Abbruch von Heartbeat = Disconnect
→ Synchronisierung mit anderen Spielern



Architektur

URL-Design

- Spiel("Match") läuft innerhalb einer Lobby
- Lobbies und Matches müssen **eindeutig** sein
- Lobby-Code `/lobby/[CODE]`
 - Beitritt mit Code (sofern angemeldet)
 - Einfache Abfolge von 7 Zeichen
- Match-UUID `/match/[UUID]`
 - Eindeutige Zuordnung ohne Dopplung
 - Ermöglicht Langzeit-Speicherung

`hexfields-studio.github.io/HexfieldsDominion/lobby/NY0L8CA`

`hexfields-studio.github.io/HexfieldsDominion/match/6bca49e2-557c-4523-b742-fc4e7f17adc0`

Architektur

Qualitäten, die wir verfolgen

Attribut	Gründe	Umsetzung
Modifiability	Neue Features einfach hinzufügen	Abstraktion in Klassen/Interfaces
Interoperability	Verbindungsabbruch erkannt	Heartbeat, Timeout-Erkennung
Usability	Intuitive Steuerung, Fehler-Feedback	Klare UI, aussagekräftige Error-Messages
Efficiency	Ressourcenschonung	Caching, Lobby-Freigabe, RAM-Cleanup
Availability	Spam-Schutz für Lobbies	Token-Caching, Request-Limits
Maintainability	Fehler-Tracking & Updates	Logging, CI/CD (GitHub Pages, Container-Neustart)

Tech-Stack

A dark blue diagonal gradient bar that starts from the bottom left and extends towards the top right, covering the lower half of the slide.

Tech-Stack

Übersicht

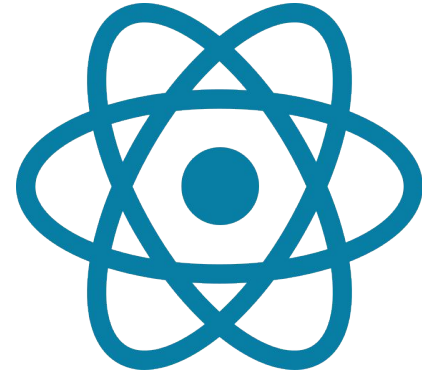
Typ	Frontend	Backend
IDE	Visual Studio Code	IntelliJ IDEA
Package Manager	bun	gradle
Framework	React TypeScript	Java Spring Boot
Tools	eslint	Docker, SonarQube, MetricsTree
Plattform	GitHub Pages	Render
Wichtigste Bibliotheken	React (Router, DOM, Select), Sass, Konva, JWT-decode, js-confetti	Spring Boot (Web, Security, Data JPA), JWT, Lombok
Sonstiges	Neon (Datenbank-Host), Jira (Projektmanagement)	

Plattformen

- GitHub
 - Git-Integration
 - Repositories
 - Actions (CI/CD)
- GitHub Pages
 - Frontend
- Render
 - Backend
- Neon
 - Datenbank

React

- Framework für Frontend-Entwicklung
- Erweiterbar mit vielen Libraries

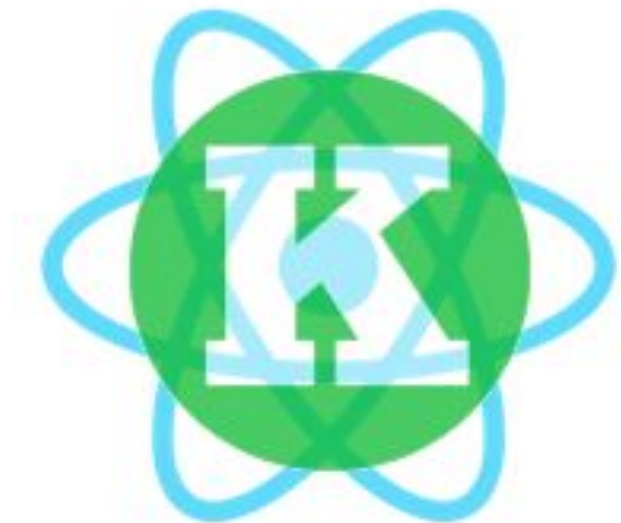


react-konva

→ High-Performance Canvas Framework

Erleichtert:

- User-Interaktion mit einem Canvas
- Rendern / Zeichnen von „Shapes“



Spring Boot

- Java Framework
- Backend
- Repositories für DB-Zugriff, REST-Controller (Endpunkte), Authentifizierung/Autorisierung, Senden von SSE



JWT Libraries

- JSON Web Token
- Frontend:
 - Auslesen Ablaufzeit
→ Rechtzeitig neuen Access-Token anfragen, um nicht authentifizierte Anfragen zu vermeiden
- Backend:
 - Prüfen Validität (abgelaufen?, enthaltener User autorisiert?)
 - Erstellung von Refresh-/Access-Token

Jira

- Projektmanagement-Tool
- Stories, Tasks, Bugs
- Kanban Board
- Zuweisung, Prioritäten, Time Tracking
- Automations: Erfasste Zeit pro
 - Person
 - Phase (per Tags)
 - Workflow (per Tags)



Qualitätssicherung

- Tests & Metriken -

Tests

- Fokus auf Backend, aktuell keine Frontend-Tests
- Unit-Tests, Integration-Tests
- JUnit, Mockito
- Abdeckung: ~36%, Ziel 80%
 - Tests für lange abgeschlossene Logik (Authentifizierung) nahezu vollständig
 - Kaum Tests für Lobby- und Spiellogik. (immer wieder Änderungen notwendig, Effizienz bei Fokus auf funktionierendem Spiel)

Metriken

- Erhebung durch Jacoco und SonarQube in der CI/CD-Pipeline
 - Testabdeckung
 - Projektmetriken und mögliche Probleme (z.B. bzgl. Wartbarkeit)
- Manuelle Erhebung durch MetricsTree-Plugin für IntelliJ
 - Halstead-Metriken (projektweit)
 - Im Vergleich zum Blog oft drei-/viermal so viel (z.B. Errors 14 statt 4)
 - Depth-Of-Inheritance-Tree (Klassen)
 - >80% gut, max. 6 (1 Klasse)
 - McCabe'sche Zyklomatische Komplexität (Methoden)
 - gut (<20): 1 Methode 16, sonst <10

CI/CD

- Infrastruktur -

CI/CD-Pipeline

Frontend

- Auslöser: Push auf main & Pull Requests
- Stages:
 - Build (TypeScript, Deps., Artifact)
 - Test (Type-Check, Tests*)
 - Deploy (GitHub Pages)
- Besonderheit: Kein Client-Update nötig, da Client im Browser läuft

* Tests aktuell nicht implementiert



HOST YOUR WEBSITE ON

GitHub Pages

CI/CD-Pipeline

Backend

- Auslöser: Push auf main & Pull Requests
- Stages:
 - Build (Gradle, Permissions, Artifact)
 - Test (JUnit, Mockito, Jacoco, SonarQube)
- Automatisches Deployment auf Render
 - Täglicher Cron-Job zum Update des Docker-Container



Quelle: <https://cdn.sanity.io/files/hvk0tap5/production/40134cba8baae95e988e65d2a0675c2521e9c731.zip>

Projektmanagement

- Statistiken -

A dark blue, diagonal shape that starts from the bottom left and extends towards the top right, covering the lower half of the slide.

Arbeitszeit

Person	Erfasste Zeit	Hauptbeitrag
Alexander Horst	2d 16h 36m	Match Seite
Marcel Steinle	2d 12h 30m	Design & Dokumente
Jona Lauber	2d 18h 5m	Auth, Dialogs, Trading

7d 23h 11m ⇔ 191h ⇔ 4,7 Arbeitswochen

Stunden pro Workflow

Workflow	Erfasste Zeit
Requirements	1d
Analysis & Design	15h 30m
Implementation	6d 11h 10m
Test	8h 55m
Deployment	5h 20m
Configuration & Change Mgmt.	4h 30m
Project Management	9h 10m
Environment	5h 41m

Stunden pro Phase

Phase	Erfasste Zeit
Inception	1d 10h 40m
Elaboration	15h 45m
Construction	7d 9h 18m
Transition	0m (nicht gestartet)

Entwicklung der Work-Items



Retrospektive

- Was haben wir gelernt? -

A dark blue, diagonal shape that starts from the bottom left corner and extends towards the top right, covering the lower half of the slide. It has a smooth, slightly curved edge.

Retrospektive

✓ Weiter so

- Pull Requests für Reviews
- Regelmäßige Kommunikation
- Häufiger Austausch über anstehende Aufgaben (wöchentliche Plannings)
- Guidelines für Branch-/Dateibenennung
- Gegenseitige Hilfe bei Fragen
- Bestreben, alle Aufgaben zu erfüllen
- Tools zu Nutze gemacht (Jira Automations)

✗ Stopp

- Ohne Setzen von Zeitplänen / Meilensteinen arbeiten
- Fehlende Synchronisation bzgl. PR Status
- Abschweifen vom Minimum-Viable-Product bzw. "Feature-Creep"

Live-Demo



